Clasificación automática de 104 tipos de flores con redes neuronales profundas*

Angel Eduardo-Luna[a,1], Andrea Estefanía De la Peña Contreras[a,2], Jannette Cristina Flores Rodríguez[a,3], Rey Iván de la Fuente Chávez[a,4] and Andrés Huerta Robinson[a,5]

[a]*Escuela de Ingeniería y Ciencias, Tecnológico de Monterrey, Monterrey, Nuevo León, México*

INFORMACIÓN

Palabras clave:
Clasificación de imágenes
Redes neuronales convolucionales
Regularización
F1 macro
Flores

Resumen

Se comparan una red totalmente conectada de referencia, una CNN original de cuatro convoluciones, una variante con dos convoluciones adicionales y una intervención de regularización con AdamW y ajuste de tasa. Se utilizan 12,753 imágenes reales de entrenamiento y 3,712 de validación de 104 categorías de flores, con entrada 112×112. Se incluyen representaciones de las arquitecturas, curvas de entrenamiento y validación, matrices de confusión y predicciones ilustrativas. El modelo elegido por F1 macro de validación es CNN + AdamW + LR, con F1 macro 0.5745 y exactitud 64.44 %. La red totalmente conectada es una referencia reconstruida, no un checkpoint histórico identificado del Avance 3. Los resultados corresponden a una sola semilla y no se presentan como evaluación sobre prueba independiente.

1. Resumen y objetivos

Este avance compara una red totalmente conectada de referencia con la CNN original del equipo y con una segunda CNN que agrega dos capas convolucionales. Además, evalúa una intervención de regularización sobre la arquitectura original. El objetivo es medir si extraer patrones espaciales mejora la clasificación de 104 categorías de flores y si aumentar la profundidad o regularizar mejora la generalización. Se reportan exactitud, F1 macro, pérdida y matrices de confusión sobre la misma partición de validación.

Para responder a la observación sobre las representaciones de los modelos, se incorporan diagramas de arquitectura con dimensiones, curvas comparables de entrenamiento y validación, y ejemplos de aciertos y errores. El notebook adjunto contiene las llamadas de entrenamiento y las salidas de la evaluación.

La red totalmente conectada se reconstruye para esta comparación a partir del cargador y del ciclo de entrenamiento existentes. No se encontró en el repositorio un modelo totalmente conectado identificado como la entrega del Avance 3; por ello no se atribuyen a aquella entrega los resultados de esta referencia.

2. Contexto del problema

Muchas categorías de flores son visualmente parecidas y distinguirlas puede requerir conocimiento especializado. Identificarlas de forma manual es lento cuando se tienen miles de fotografías. Por eso resulta útil contar con modelos que reconozcan la clase de una flor a partir de una imagen.

Este proyecto parte de la competencia "Petals to the Metal", encontrado en Kaggle, cuyo reto es construir un modelo que identifique el tipo de flor en un conjunto de imágenes. La competencia se limita a 104 clases de flores, y consiste en un problema de clasificación multiclase de imágenes en el que el modelo debe tener la capacidad de identificar valores finos como la forma, estructura y color para distinguir entre categorías cercanas.

Este problema tiene varias dificultades que lo hacen relevante desde un enfoque de redes neuronales. Primeramente, las clases son heterogéneas, ya que algunas de ellas solo tienen un subtipo de flor y otras agrupan muchas subvariedades. Además, las imágenes de las flores no fueron capturadas en el mismo entorno, dando más variedad a las mismas. Por último, el número de imágenes por cada clase tampoco es uniforme, por lo que, considerando la métrica de evaluación, es un punto fuerte a tomar en cuenta que el modelo funcione de manera efectiva tanto para clases con pocos ejemplos como también para aquellas con muchos.

3. Descripción del dataset

3.1. Origen

Los datos vienen de *Petals to the Metal – Flower Classification on TPU*, una competencia *Getting Started* de Kaggle cuyo objetivo es aprender a entrenar modelos en TPU (Kaggle, s.f.). Según la descripción de la competencia, las imágenes se tomaron de cinco conjuntos de datos públicos. Algunas clases son muy específicas (por ejemplo, *pink primrose*, que contiene un solo subtipo de flor) y otras agrupan muchas subvariedades (por ejemplo, *wild rose*).

Al revisar la lista de clases nos dimos cuenta de que las primeras 102 siguen exactamente el orden del conjunto Oxford 102 Flowers (Nilsback y Zisserman, 2008): 94 conservan el mismo nombre y las otras 8 se renombraron, se corrigieron o se ampliaron (por ejemplo, *oxeye daisy* pasó a ser *daisy* y *bearded iris* pasó a ser *iris*). Las dos clases extra

* Situación problema: Red convolutiva, Avance 4. Diseño de Redes Neuronales y Aprendizaje Profundo (TC2035.201). Basado en la competencia *Petals to the Metal – Flower Classification on TPU* (Kaggle).

[1] Matrícula: A00842657
[2] Matrícula: A00840973
[3] Matrícula: A01255711
[4] Matrícula: A01709445
[5] Matrícula: A00838626

Tabla 1
Particiones del conjunto de datos.

Partición	Imágenes	%	£Etiquetado?
Entrenamiento	12,753	53.5	Sí
Validación	3,712	15.6	Sí
Prueba	7,382	31.0	No
Total	23,847	100.0	

son *common tulip* y *wild rose*. Las etiquetas están en inglés, así que usamos los nombres originales en cursiva.

3.2. Tamaño y estructura

En total hay 23,847 imágenes a color (RGB) de 104 clases, repartidas como muestra la Tabla 1. Vienen en formato TFRecord, que junta muchas imágenes en pocos archivos binarios para que la TPU las lea rápido. Cada registro tiene tres campos: `image` (la foto codificada en JPEG), `id` (un identificador único) y `class` (la etiqueta, un entero de 0 a 103). Los registros de prueba no traen `class`, porque esas etiquetas solo las tiene Kaggle para calificar.

Cada partición viene en 4 resoluciones: 192×192, 224×224, 331×331 y 512×512 píxeles, con 16 archivos TFRecord por partición, por lo que la base de datos trae 192 archivos TFRecord (4 resoluciones × 3 particiones × 16 archivos) más `sample_submission.csv`. Tener varias resoluciones nos conviene porque no todas las arquitecturas se diseñaron para el mismo tamaño de entrada: ResNet50, DenseNet201 y EfficientNetB0 hacen uso de 224 × 224, mientras que las EfficientNet más grandes usan resoluciones mayores. Para dimensionar el problema, una sola imágen de 224 × 224 ya tiene 150,528 valores.

En las ejecuciones de este avance se leen los TFRecord de 224×224, pero se redimensionan las imágenes a 112×112 antes de entrar a cualquiera de los modelos. La resolución de origen y la entrada efectiva de entrenamiento son, por tanto, distintas. Se conservan las particiones y etiquetas originales.

3.3. Ejemplos de las imágenes

La tabla 2 muestra algunas clases concretas y lo que cada una nos dice sobre el problema. Además, dentro de una misma clase, las fotos cambian mucho de escala, pose e iluminación (Nilsback and Zisserman, 2008)

La Figura 1 muestra fotos reales del conjunto de entrenamiento.

3.4. Balance de clases y métrica de evaluación

Con 12,753 imágenes de entrenamiento y 104 clases, tenemos en promedio 123 imágenes por clase (12,753/104 ≈ 122.6). Sin embargo, las clases no tienen el mismo tamaño: como unas clases agrupan muchas subvariedades y otras solo una, algunas tienen muchas más imágenes que otras.

Por este desbalance, la exactitud no basta: un modelo puede acertar mucho en las clases grandes, fallar en las clases pequeñas y aún verse bien. Por esto mismo, Kaggle califica con el F1 macro (Kaggle, s.f.). Para cada clase c se calcula la precisión P_c, la sensibilidad (*recall*) R_c y su media armónica:

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \tag{1}$$

$$F1_c = \frac{2\,P_c\,R_c}{P_c + R_c}, \qquad F1_{\text{macro}} = \frac{1}{104}\sum_{c=0}^{103} F1_c, \tag{2}$$

donde TP_c, FP_c y FN_c son los verdaderos positivos, falsos positivos, y falsos negativos de la clase c . Así todas las clases pesan lo mismo, sin importar cuántas imágenes tengan.

3.5. Implicaciones para el proyecto

Como Kaggle ya separa entrenamiento y validación, no necesitamos hacer nuestra propia partición: entrenaremos con `train`, elegiremos modelos e hiperparámetros con `val` y usaremos `test` solo para la predicción final. Las imagenes vienen en JPEG con valores de 0 a 255, así que habrá que decodificarlas y escalarlas (a [0, 1] o con el preprocesamiento que pida cada red preentrenada). Como las etiquetas son enteros, la red producirá 104 valores de salida, uno por cada clase, que podrán transformarse en probabilidades mediante Softmax. El entrenamiento se realizará utilizando entropía cruzada categórica dispersa.

4. Algoritmos y aplicaciones

Dado el problema planteado, su solucion implica el uso de algoritmos de vision por computadora.

4.1. Redes Neuronales Convolucionales (CNNs)

Las redes neuronales convolucionales (CNN) son una alternativa especialmente adecuada para este problema debido a su capacidad para procesar información espacial. Mediante filtros convolucionales pueden aprender automáticamente características de las imágenes, como bordes, texturas y formas. En las capas internas pueden emplearse funciones de activación como ReLU, mientras que Softmax puede utilizarse en la salida para representar las probabilidades de las distintas clases.

Este método parece ser el principal y el mejor para producir resultados optimos, aunque, dado a la gran cantidad de datos que se tienen que tratar y la variedad de operaciones que se llevan a cabo, hay que considerar que es un metodo lento y computacionalmente pesado. Esto habilita la posibilidad de usar otros algoritmos que se ajusten a la necesidad del problema.

4.2. Otros algoritmos sin aprendizaje profundo

La Figura 3 de la sección de metodología complementa el esquema conceptual anterior con las capas y dimensiones de los modelos realmente entrenados. ResNet, AlexNet, Inception y los métodos tradicionales que se describen a continuación son alternativas de contexto; no se reportan como modelos ejecutados en esta comparación.

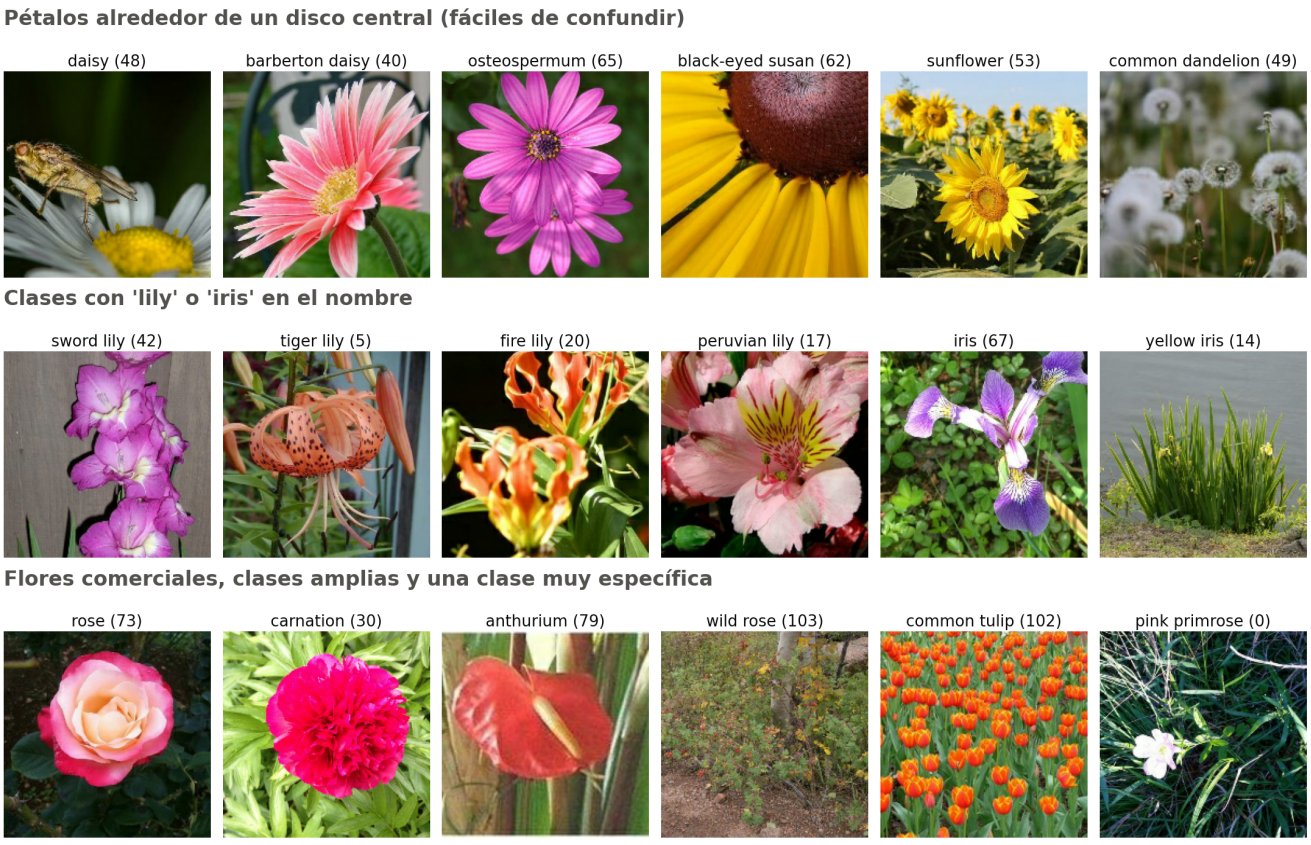


Figura 1: Ejemplos reales del conjunto de entrenamiento con su clase y número de etiqueta.

Tabla 2
Clases de ejemplo (nombre y etiqueta) más lo que ilustran.

Clase (etiqueta)	Que ilustra
sunflower (53)	Girasol. Flor grande y muy característica. Posiblemente sea de las más fáciles de identificar.
daisy (48), *barberton daisy* (40), *osteospermum* (65), *black-eyed susan* (62)	Margarita, gerbera, osteospermum y rudbeckia. Todas tienen pétalos alrededor de un disco central. Pueden surgir confusiones entre ellas.
sword lily (42), *tiger lily* (5), *fire lily* (20), *peruvian lily* (17)	Gladiola, lirio tigre, gloriosa y alstroemeria. Todas dicen *lily*, pero son de cuatro familias botánicas distintas: el modelo tiene que fijarse en la forma, no en el nombre.
iris (67), *yellow iris* (14)	Misma forma general, la diferencia principal es el color.
rose (73), *wild rose* (103)	Dos clases de rosas. *Wild rose* es amplia (varias subvariedades) y no viene de Oxford 102.
common tulip (102)	Tulipán. Clase amplia que tampoco viene de Oxford 102.
pink primrose (0)	Clase muy específica con un solo subtipo de flor.
carnation (30)	Clavel, una de las flores comerciales que se producen en el Estado de México.

k vecinos más cercanos (KNN) Cada imagen lo representa como un vector de características. Usa normalmente la distancia euclidiana para medir las similaridades entre los puntos, y compara con sus k vecinos más cercanos para clasificar la imagen. Es un algoritmo de aprendizaje supervisado.

Bosque aleatorio Se compone de un conjunto de árboles de decisión, se calcula la media de los resultados de los árboles.

Máquinas de Vectores de Soporte (SVMs) Este algoritmo encuentra la línea óptima para separar dos clases. Una variación llamada SVM multiclase existe para problemas multiclase, como este contexto.

4.3. Otros modelos de aprendizaje profundo

ResNet Conocida también como *Residual network*. Útil para entrenar redes neuronales muy profundas. La cota superior para implementar este método es la capacidad de nuestro *hardware* o acceso a equipo

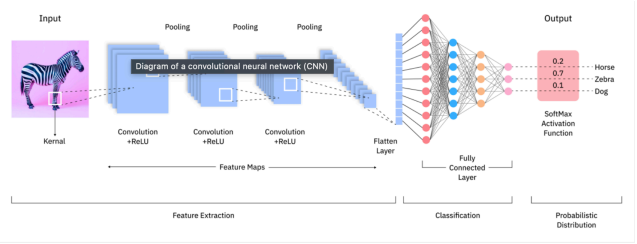


Figura 2: La convolución descompone la imagen en características para aprender a clasificar (Clark, 2025).

computacional que sea capaz de entrenar este tipo de redes más complejas.

Otros modelos ya implementados Como **AlexNet** o **GoogleNet/inception** que tienen una gran capacidad para reconocer y clasificar imágenes.(Clark, 2025)

4.4. Aplicaciones posibles

Un modelo capaz de clasificar tipos de flores a partir de imágenes puede tener distintas aplicaciones reales. Una de ellas es la **identificación automática de plantas**, donde un usuario podría tomar una fotografía y recibir como resultado la especie o clase más probable.

Un ejemplo de este tipo de aplicaciones yace en el apoyo a plataformas de ciencia ciudadana y el monitoreo de biodiversidad. Por ejemplo, iNaturalist ha acumulado más de 300 millones de observaciones hechas por personas comunes (iNaturalist, 2026) y utiliza un modelo de visión por computadora, entrenado con esas mismas observaciones, para sugerir la especia a partir de una fotografía. Sin este apoyo automático, cada observación dependería de que una persona la revisara detenidamente, lo que haría el proceso lento y difícil de escalar. Automatizar parte de la tarea ayuda a mejorar la eficiencia y aporta de gran manera a la obtención de datos sobre biodiversidad. Este proyecto es una versión a menor escala de este tipo de sistemas.

Otra aplicación sería como **herramienta educativa**, por ejemplo, en aplicaciones para estudiantes o visitantes de jardines botánicos que permitan reconocer flores y consultar información sobre ellas.

Sin embargo, el modelo desarrollado solo podría reconocer las 104 clases presentes en el conjunto de datos, por lo que una aplicación real más amplia requeriría entrenarlo con una mayor variedad de especies.

5. Metodología

5.1. Representación de las arquitecturas implementadas

La Figura 3 representa los tres diseños entrenados desde cero. La entrada efectiva es una imagen RGB de 112×112 píxeles, obtenida de los TFRecord de 224×224 mediante redimensionamiento bilineal con antialias. Esto corrige la descripción anterior, que presentaba 224×224 como resolución de entrenamiento.

La red **totalmente conectada de referencia** aplana los $112 \times 112 \times 3$ valores de entrada en un vector de 37,632 componentes. Aplica una capa densa de 256 unidades con ReLU, dropout de 0.5 y una salida de 104 logits. No contiene convoluciones ni pooling. Es una comparación reconstruida, como se aclara en la sección 1, y utiliza el mismo cargador, aumentos y ciclo de entrenamiento que la CNN.

La **CNN original** emplea cuatro convoluciones de 3×3, padding *same*, ReLU y 32, 64, 128 y 256 filtros. Cada convolución va seguida de max pooling de 2×2. Las salidas de esos bloques son $56 \times 56 \times 32$, $28 \times 28 \times 64$, $14 \times 14 \times 128$ y $7 \times 7 \times 256$. Después aplica global average pooling (GAP), una capa densa de 256 unidades con ReLU, dropout de 0.5 y 104 logits de salida.

La **CNN ampliada** agrega dos capas convolucionales ocultas: una de 128 filtros después del tercer pooling y otra de 256 después del cuarto. Ambas usan kernel 3×3, padding *same* y ReLU, sin pooling adicional. Por tanto, esta versión tiene seis convoluciones. No se trata de dos capas densas nuevas.

La convolución comparte los pesos del filtro entre posiciones y analiza vecindarios de píxeles. Así puede aprender bordes, texturas y partes de pétalos antes de combinarlos. Una capa densa sobre píxeles aplanados asigna pesos independientes a cada posición y no impone esta estructura local. Esta diferencia motiva la comparación, pero la mejora debe comprobarse con las métricas.

5.2. Preprocesamiento y aumento de datos

Se decodifica cada JPEG como RGB, se convierten sus valores a float32, se normalizan mediante $x_{\text{norm}} = x/255$ y se redimensionan a 112×112. Las etiquetas son enteros entre 0 y 103. Se mantiene la separación de archivos original entre entrenamiento y validación, sin incorporar imágenes de validación al entrenamiento.

Solo en entrenamiento se aplican volteo horizontal, rotación aleatoria de factor 0.05, zoom de 0.1 y contraste de 0.1. Los valores resultantes se recortan a $[0, 1]$. Los aumentos se calculan después de la caché y del mezclado, por lo que cambian entre épocas. La validación usa imágenes deterministas sin aumentos.

5.3. Pérdida y actualización de parámetros

Los modelos producen 104 logits z_k. La entropía cruzada se calcula directamente sobre ellos, sin añadir una segunda softmax a la pérdida:

$$L = -\log\left(\frac{\exp(z_c)}{\sum_{k=1}^{104}\exp(z_k)}\right), \tag{3}$$

donde c es la clase verdadera. Para visualizar la confianza se transforma la salida mediante softmax; esta confianza no constituye una evaluación de calibración.

Ambas CNN sin la intervención adicional usan Adam con tasa inicial 10^{-3}. La referencia totalmente conectada utiliza Adam con 10^{-4} después de que un primer intento con 10^{-3} colapsó a predecir una sola clase. Todos los modelos utilizan lotes de 64 imágenes. La variante regularizada usa

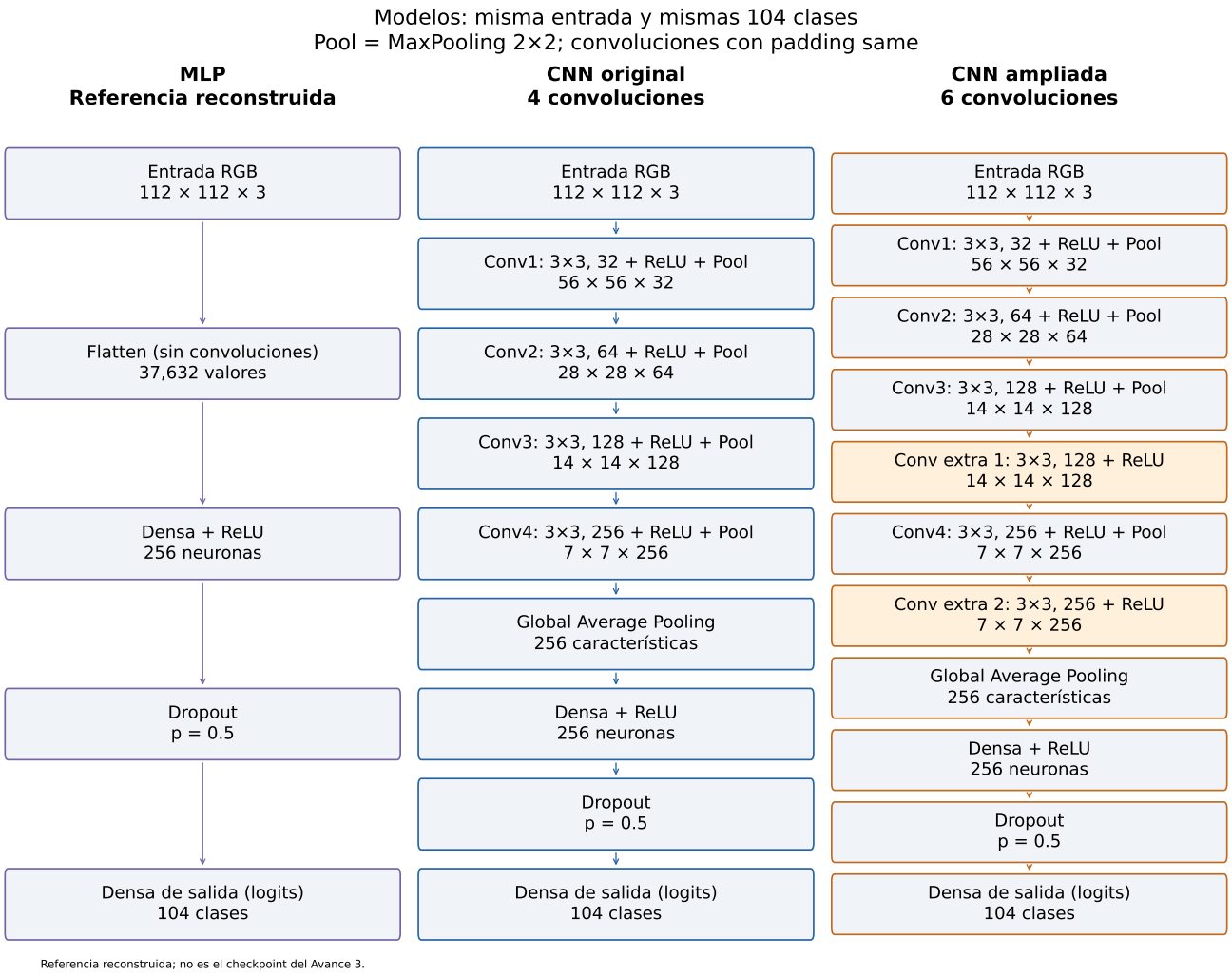


Figura 3: Representación de la red totalmente conectada de referencia, la CNN original y la CNN con dos convoluciones adicionales. Las dimensiones corresponden a las salidas de cada etapa; las capas adicionales se destacan en naranja. La variante regularizada conserva la arquitectura de la CNN original.

AdamW y un ajuste de tasa que se detalla en la sección 7. Se entrena con PyTorch/DirectML en la GPU AMD Radeon RX 6700S, conservando el cargador y los aumentos de TensorFlow. El mejor checkpoint se exporta a Keras y se comprueba que mantiene los logits y las predicciones sobre un lote de control.

5.4. Imágenes sintéticas: experimento independiente

El proyecto también incluye destilación de datos mediante Distribution Matching (DM). IPC 1 significa una imagen sintética por cada una de las 104 clases, es decir, 104 imágenes; IPC 10 significa 1,040. El generador utiliza el extractor convolucional de la arquitectura original con pesos aleatorios que se renuevan durante la optimización de los píxeles. No utiliza como generador el checkpoint entrenado de aproximadamente 63 % de exactitud. Estas figuras históricas se conservan separadas; los experimentos principales de este avance entrenan con las 12,753 imágenes reales y no se atribuyen sus métricas a los conjuntos sintéticos.

6. Protocolo experimental y correcciones

6.1. Condiciones de comparación

Se usan los mismos 12,753 ejemplos reales de entrenamiento y 3,712 de validación, 104 clases, entrada 112×112, lote de 64 y semilla 0. Se permiten hasta 150 épocas. Se guarda el checkpoint con el mayor F1 macro de validación; por ello, la exactitud final reportada corresponde a ese mismo checkpoint y no al máximo independiente de exactitud. Las dos CNN comparadas por profundidad comparten optimizador, aumentos, dropout y paciencia de cinco evaluaciones sin mejora. La referencia totalmente conectada conserva los mismos datos, aumentos y dropout, pero usa una tasa menor (10^{-4}), paciencia de diez evaluaciones y un mínimo de diez épocas, tras diagnosticar el colapso de su primer intento con 10^{-3}. Ese intento se conserva en la carpeta

de diagnósticos; no se oculta como si fuera una ejecución inexistente. La intervención combinada de regularización permite una paciencia mayor y cambia el optimizador y su tasa; no es una ablación aislada de cada técnica.

Las curvas contienen métricas de entrenamiento con aumentos y dropout activos, mientras que la validación se calcula sin ellos. Por esta razón, una exactitud de validación mayor que la de entrenamiento no implica por sí sola fuga de datos. Para diagnosticar sobreajuste se examinan conjuntamente la tendencia de la pérdida, la exactitud y F1 macro, sin usar únicamente la distancia entre dos curvas.

La comparación se limita a una semilla y utiliza validación para seleccionar épocas e hiperparámetros. No proporciona intervalos de confianza ni una medida final independiente de generalización. El conjunto de prueba de Kaggle carece de etiquetas locales; no se presentan los resultados de validación como exactitud de prueba ni como puntaje de la competencia.

La semilla y las configuraciones se guardan para repetir el procedimiento, pero las operaciones de GPU y la canalización paralela no garantizan resultados idénticos bit a bit. Las métricas históricas y las nuevas ejecuciones se identifican por separado en sus respectivas carpetas.

6.2. Trazabilidad y ajustes del documento

Las ejecuciones nuevas se guardan bajo `reports/avance4_2026-10-21`, con configuraciones, históricos por época, checkpoints y predicciones por imagen. Las figuras se exportan tanto a PNG para presentación como a PDF para el reporte. Un manifiesto registra la versión del código y sus hashes; los experimentos anteriores se conservan y no se mezclan con las métricas nuevas.

Respecto del documento anterior, las secciones 1 a 6 ahora explicitan los objetivos medibles, la resolución realmente usada y el lote de 64; representan las arquitecturas y la ubicación exacta de las dos capas nuevas; distinguen la referencia totalmente conectada de la CNN original; separan destilación y clasificación; y aclaran el criterio de selección y el alcance de la validación. Se atiende la observación compartida sobre las representaciones de los modelos. No se dispone del texto completo de la retroalimentación del Avance 3, por lo que no se afirma haber atendido observaciones adicionales no documentadas.

7. Regularización

7.1. Diagnóstico con curvas de entrenamiento y validación

La Figura 4 compara exactitud y entropía cruzada para cada modelo. Se muestran todas las épocas ejecutadas y se marca la época seleccionada por F1 macro. No se confunde la última época con el checkpoint final evaluado. En una fase de sobreajuste, el modelo sigue reduciendo su error de entrenamiento sin obtener la misma mejora en validación; en particular, una pérdida de validación que aumenta revela errores más confiados aunque la exactitud cambie poco.

- **MLP de referencia**: la menor pérdida de validación fue 3.339 en la época 76. Al terminar, la pérdida de entrenamiento fue 3.595 y la de validación 3.345; la exactitud de entrenamiento fue 11.78 % y la de validación 15.41 %. Estas curvas deben interpretarse junto con F1 macro; no se concluye sobreajuste solo por una diferencia entre exactitudes.

- **CNN original**: la menor pérdida de validación fue 1.520 en la época 53. Al terminar, la pérdida de entrenamiento fue 0.937 y la de validación 1.609; la exactitud de entrenamiento fue 71.17 % y la de validación 62.53 %. La reducción de la pérdida de entrenamiento junto con una pérdida de validación mayor que su mínimo es evidencia de sobreajuste en la fase final.

- **CNN + 2 convoluciones**: la menor pérdida de validación fue 1.658 en la época 31. Al terminar, la pérdida de entrenamiento fue 0.564 y la de validación 2.024; la exactitud de entrenamiento fue 82.66 % y la de validación 60.48 %. La reducción de la pérdida de entrenamiento junto con una pérdida de validación mayor que su mínimo es evidencia de sobreajuste en la fase final.

- **CNN + AdamW + LR**: la menor pérdida de validación fue 1.494 en la época 52. Al terminar, la pérdida de entrenamiento fue 0.785 y la de validación 1.512; la exactitud de entrenamiento fue 76.16 % y la de validación 64.41 %. Estas curvas deben interpretarse junto con F1 macro; no se concluye sobreajuste solo por una diferencia entre exactitudes.

7.2. Técnicas aplicadas y efecto medido

Todos los modelos usan dropout de 0.5 antes de la salida, aumentos de datos solo en entrenamiento y parada temprana con restauración del mejor checkpoint. No se entrenó una ablación sin dropout ni sin aumentos; por eso no se atribuye por separado una mejora numérica a estas dos técnicas.

La intervención adicional conserva la CNN de cuatro convoluciones, pero utiliza AdamW con decaimiento de pesos de 0.01 sobre matrices de pesos, excluyendo sesgos. Se trata de decaimiento desacoplado y no de añadir un término L2 a la pérdida mostrada. Además, reduce la tasa de aprendizaje a la mitad cuando la pérdida de validación no mejora durante dos evaluaciones, hasta un mínimo de 10^{-5}. La paciencia de parada temprana pasa de 5 a 12 evaluaciones para permitir actuar a esa reducción. Esta es una intervención combinada: no permite aislar cuánto aportó el decaimiento, el ajuste de tasa o la paciencia.

Frente a la CNN original, la variante regularizada cambia la exactitud de 63.25 % a 64.44 % (+1.19 puntos porcentuales) y F1 macro de 0.5740 a 0.5745 (+0.0005). Ambos valores se obtienen de los respectivos checkpoints seleccionados por F1 macro sobre la misma validación.

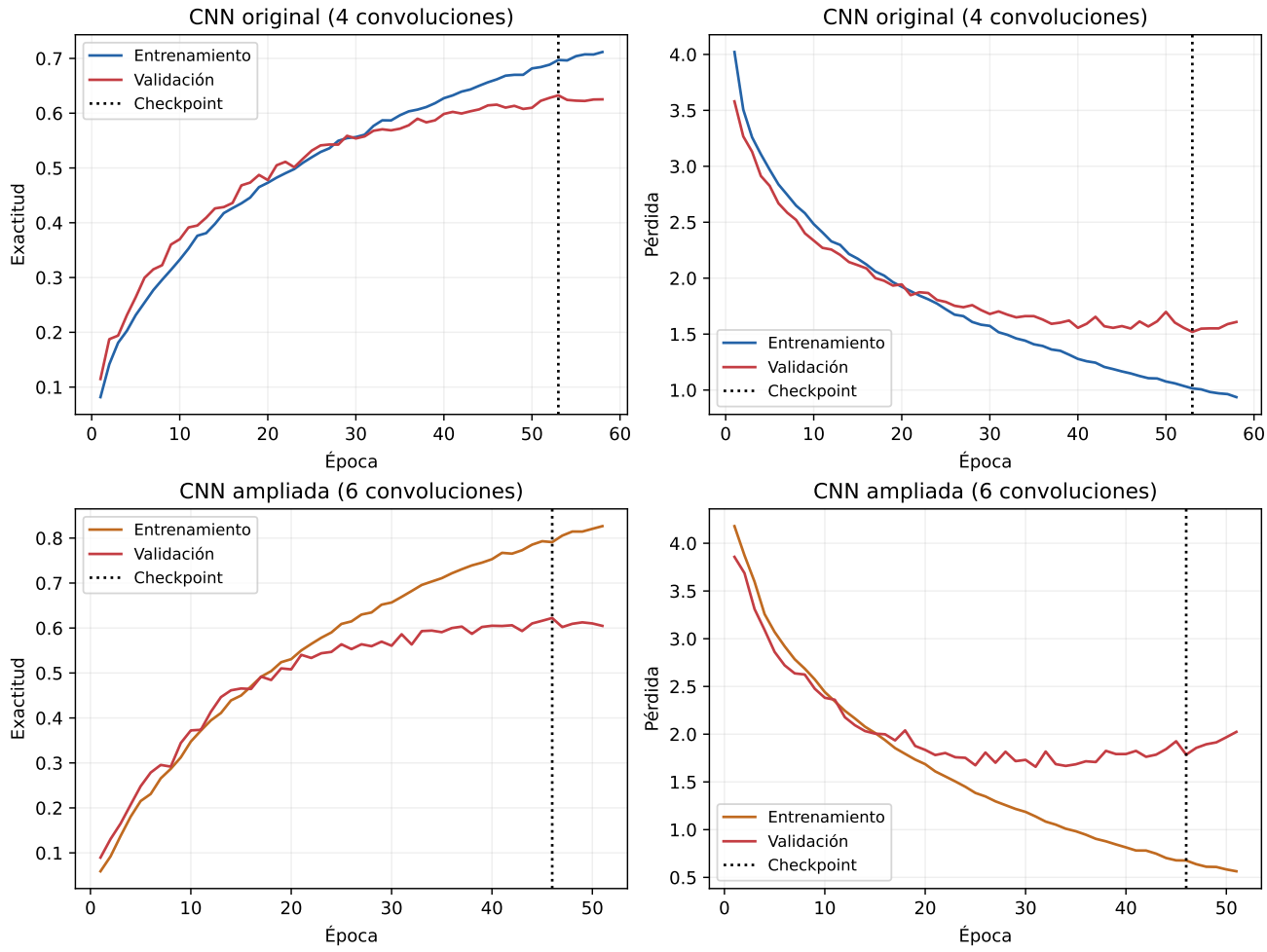


Figura 4: CNN original y CNN ampliada: curvas reales por época. El entrenamiento usa aumentos y dropout; la validación no. Las líneas verticales marcan el mejor checkpoint por F1 macro.

La diferencia de F1 macro es pequeña. Con una sola semilla no se considera evidencia de una mejora robusta o estadísticamente significativa.

Una mejora de exactitud acompañada de una caída de F1 macro no se interpreta como mejora general: el criterio prioritario es el desempeño equilibrado entre las 104 clases. Tampoco se afirma que el sobreajuste desaparezca; se compara la evidencia de las curvas y se conserva el checkpoint que generaliza mejor según la validación disponible.

8. Resultados y evaluación

8.1. Comparación de modelos

La Tabla 3 contiene exclusivamente las nuevas ejecuciones de este avance. Todos los resultados se calcularon sobre 3,712 imágenes reales de validación, sin aumentos, y corresponden a un checkpoint identificado. Se conservan los históricos de entrenamiento y las predicciones por imagen.

8.2. Puente entre la red totalmente conectada y la CNN

La CNN original obtuvo 63.25 % de exactitud frente a 15.52 % de la red totalmente conectada de referencia (+47.74 puntos porcentuales). F1 macro cambió de 0.0280 a 0.5740. La MLP tiene 9,660,776 parámetros frente a 480,936 de la CNN.

El resultado es compatible con la ventaja de compartir filtros y preservar vecindarios de píxeles: una CNN puede reutilizar patrones locales sin asignar un peso independiente a cada píxel de cada posición. GAP también limita los parámetros de la etapa densa. Esta explicación es una interpretación arquitectónica del resultado, no una prueba causal aislada; ambas redes tienen distinta capacidad y pueden requerir otros hiperparámetros. Si la MLP presenta baja exactitud tanto en entrenamiento como en validación, su limitación es de ajuste u optimización y no se debe diagnosticar como sobreajuste solo porque rinda peor que la CNN.

La comparación solicitada con el modelo exacto entregado en el Avance 3 queda limitada por la ausencia de ese artefacto identificado. El experimento reconstruido permite

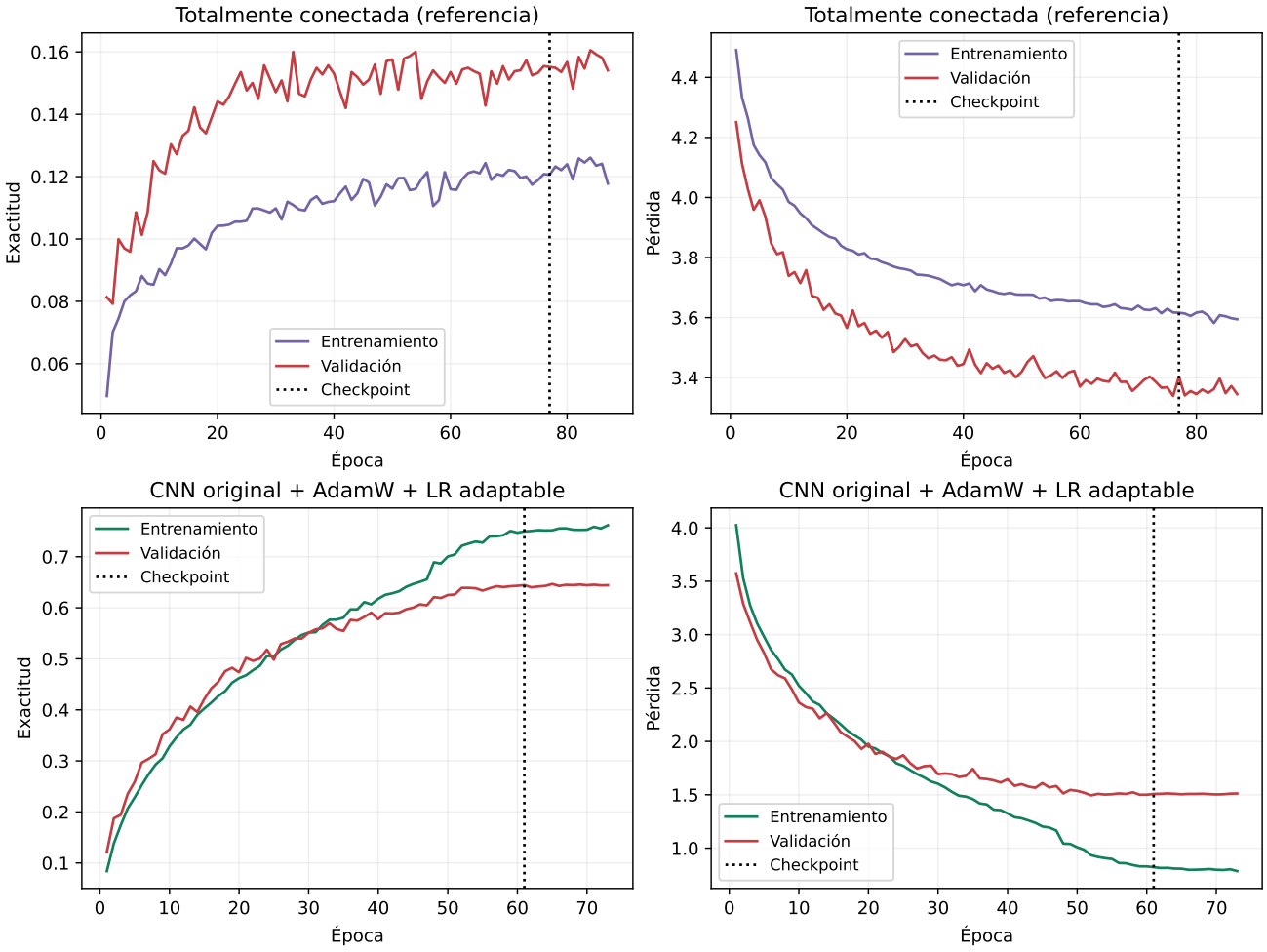


Figura 5: Curvas de la referencia totalmente conectada y la CNN con la intervención combinada de regularización. Se mantienen separados sus checkpoints y métricas.

Tabla 3
Resultados nuevos; el checkpoint de cada modelo se selecciona por F1 macro.

Modelo	Parámetros	Época	Exactitud (%)	F1 macro	Pérdida	Tiempo (s)
MLP de referencia	9,660,776	77	15.52	0.0280	3.403	336.7
CNN original	480,936	53	63.25	0.5740	1.520	525.8
CNN + 2 convoluciones	1,218,600	46	62.23	0.5502	1.786	500.6
CNN + AdamW + LR	480,936	61	64.44	0.5745	1.507	605.6

Época: checkpoint seleccionado, no número total de épocas ejecutadas. Tiempo: entrenamiento, excluyendo evaluación y generación de figuras. La MLP es una referencia reconstruida; no un resultado histórico atribuible al Avance 3. Una sola semilla por modelo.

comparar familias de modelos en condiciones comunes, sin inventar resultados históricos. El primer intento con Adam a 10^{-3} predijo una sola clase (5.82 % de exactitud, F1 macro 0.0011); se conserva como diagnóstico. La referencia final utiliza 10^{-4} y mayor paciencia. Esta adaptación de optimización se declara para no atribuir exclusivamente a la arquitectura todas las diferencias observadas.

8.3. Efecto de agregar dos convoluciones

La versión de seis convoluciones obtuvo 62.23 % de exactitud y F1 macro 0.5502. Frente a la CNN original, la diferencia es -1.02 puntos de exactitud y -0.0238 de F1 macro. Los parámetros aumentan de 480,936 a 1,218,600. Agregar profundidad incrementa la capacidad de representación, pero no garantiza mejorar validación: también modifica la optimización y puede facilitar el sobreajuste. La conclusión corresponde a esta configuración y semilla, sin afirmar superioridad universal de una profundidad.

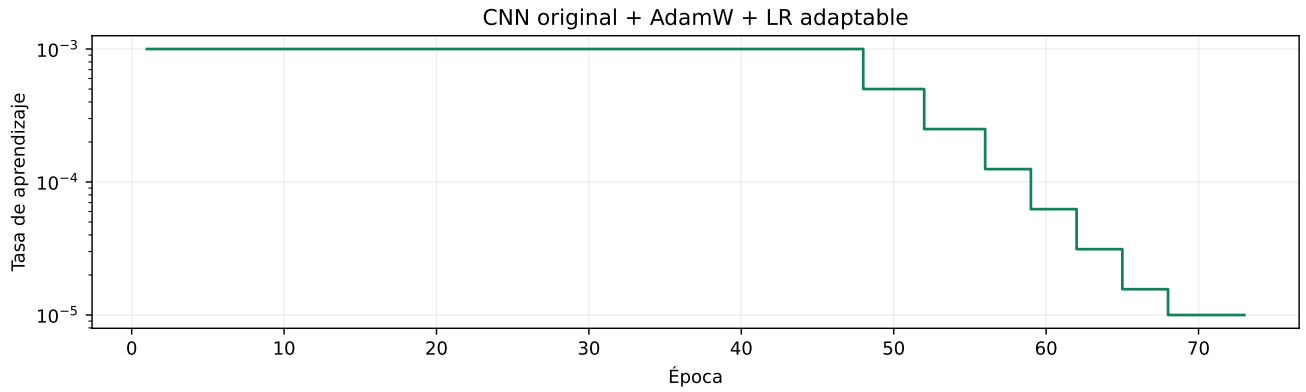


Figura 6: Tasa de aprendizaje realmente utilizada en cada época de la variante regularizada. Las reducciones responden al estancamiento de la pérdida de validación.

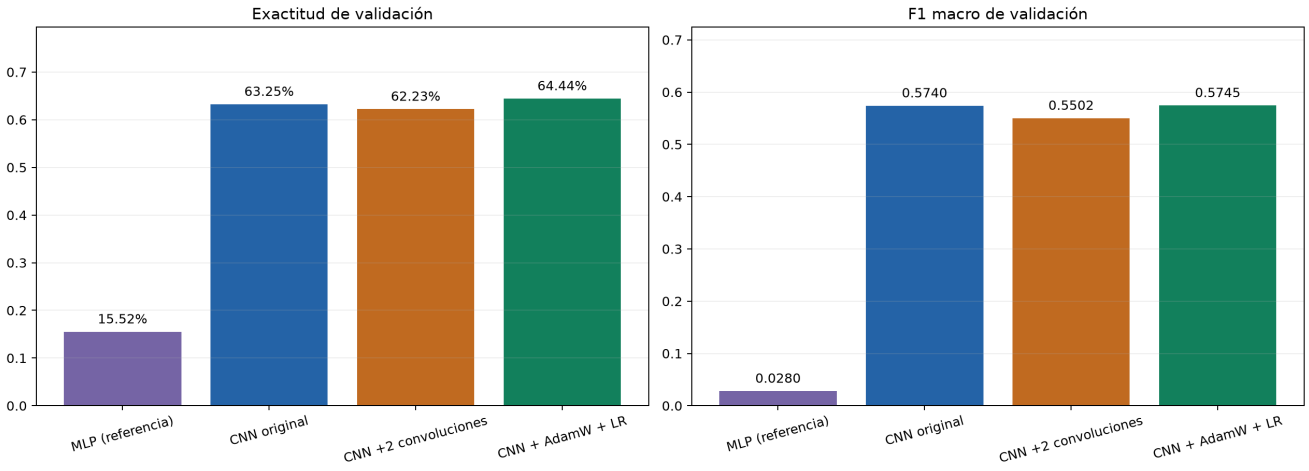


Figura 7: Exactitud y F1 macro sobre la misma validación. El modelo elegido se determina por F1 macro y puede diferir del que maximiza la exactitud.

8.4. Modelo elegido y análisis de errores

Se conserva **CNN + AdamW + LR**, por tener el mayor F1 macro de esta comparación: 0.5745, con exactitud 64.44 %, pérdida 1.507 y checkpoint en la época 61. La Figura 9 muestra la matriz de confusión normalizada por clase real. La diagonal representa recall por clase; los valores fuera de ella son errores. Se entregan además la matriz de conteos, los CSV de ambas matrices y las predicciones individuales.

Algunos de los pares más confundidos son:

- common tulip predicha como rose: 12 imágenes.
- carnation predicha como rose: 12 imágenes.
- anthurium predicha como rose: 10 imágenes.
- windflower predicha como wild rose: 8 imágenes.
- wild rose predicha como windflower: 8 imágenes.

La Figura 10 ilustra aciertos y errores del checkpoint elegido. Se muestran seis de cada tipo, seleccionados con semilla fija, por lo que esa figura no estima la tasa de error global.

8.5. Conclusiones y limitaciones

La comparación documenta el cambio de una referencia totalmente conectada a un extractor convolucional, el efecto de aumentar dos convoluciones y el efecto observado de una intervención combinada de regularización. La elección se basa en F1 macro para evitar que las clases numerosas dominen la decisión. Las curvas permiten diagnosticar el comportamiento durante el entrenamiento y justifican conservar checkpoints anteriores al final cuando la validación deja de mejorar.

La validación se utiliza también para selección, los resultados son de una semilla y no se dispone de etiquetas de prueba locales. Harían falta varias semillas, ablaciones controladas y una evaluación independiente para cuantificar incertidumbre y separar el efecto de cada técnica.

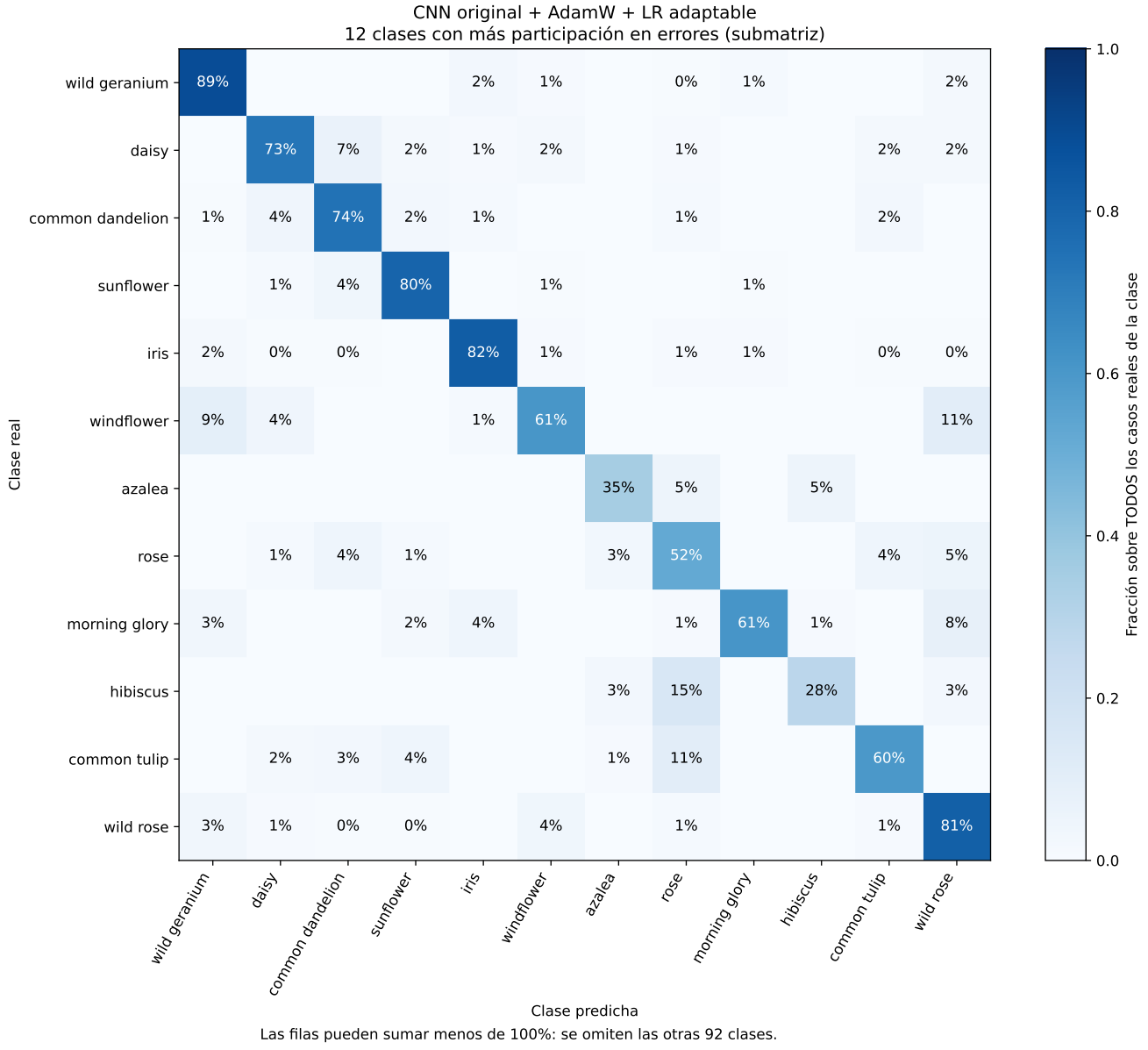


Figura 8: Vista legible de las 12 clases con mayor participación en errores. Es una submatriz de la matriz completa: los porcentajes se calculan sobre todos los ejemplos reales de cada clase y las filas mostradas pueden sumar menos de 100 %.

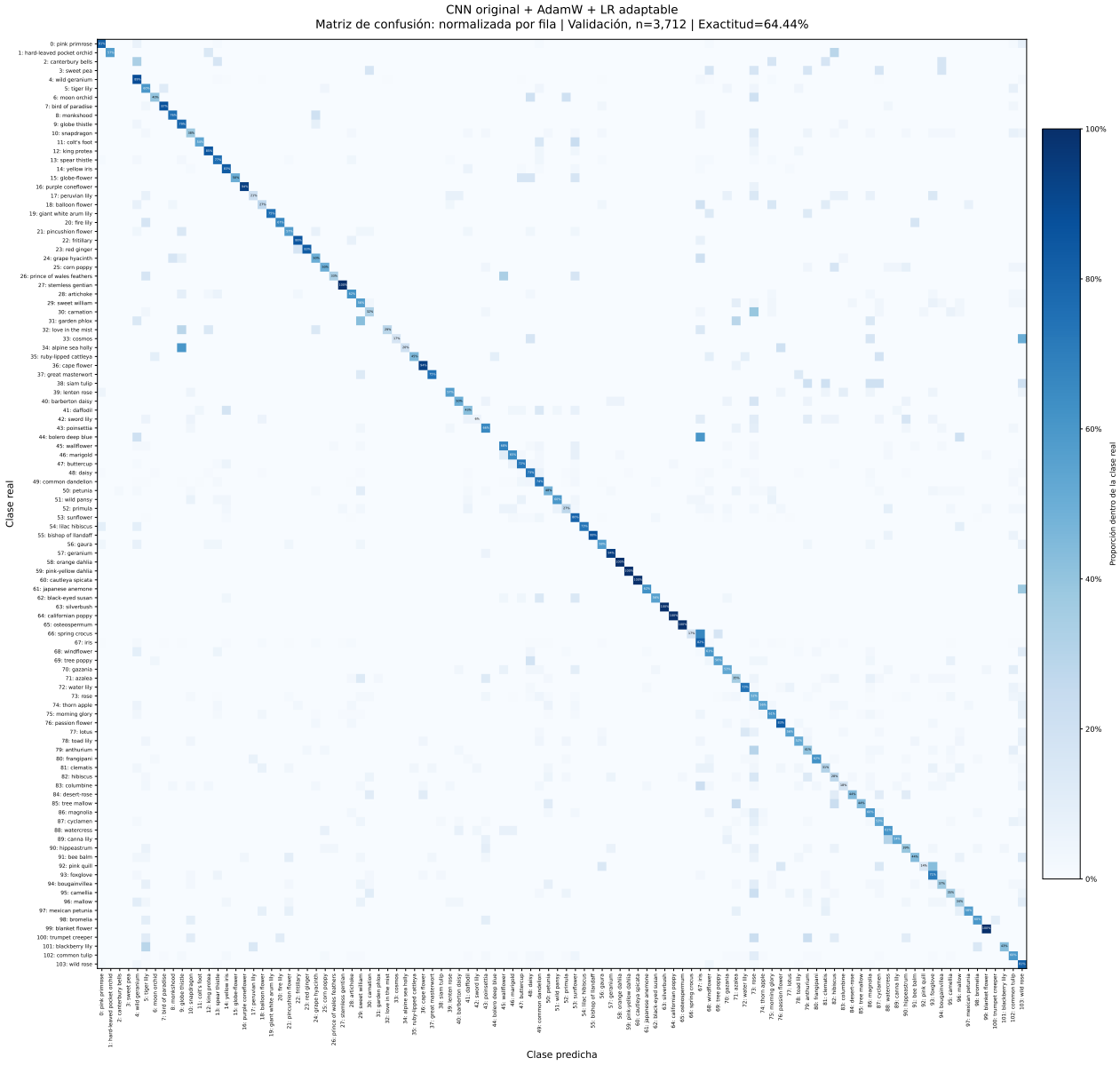


Figura 9: Matriz de confusión del modelo elegido sobre validación. Filas: clase real; columnas: clase predicha. Normalización por fila. Se conserva la versión de alta resolución con las 104 clases.

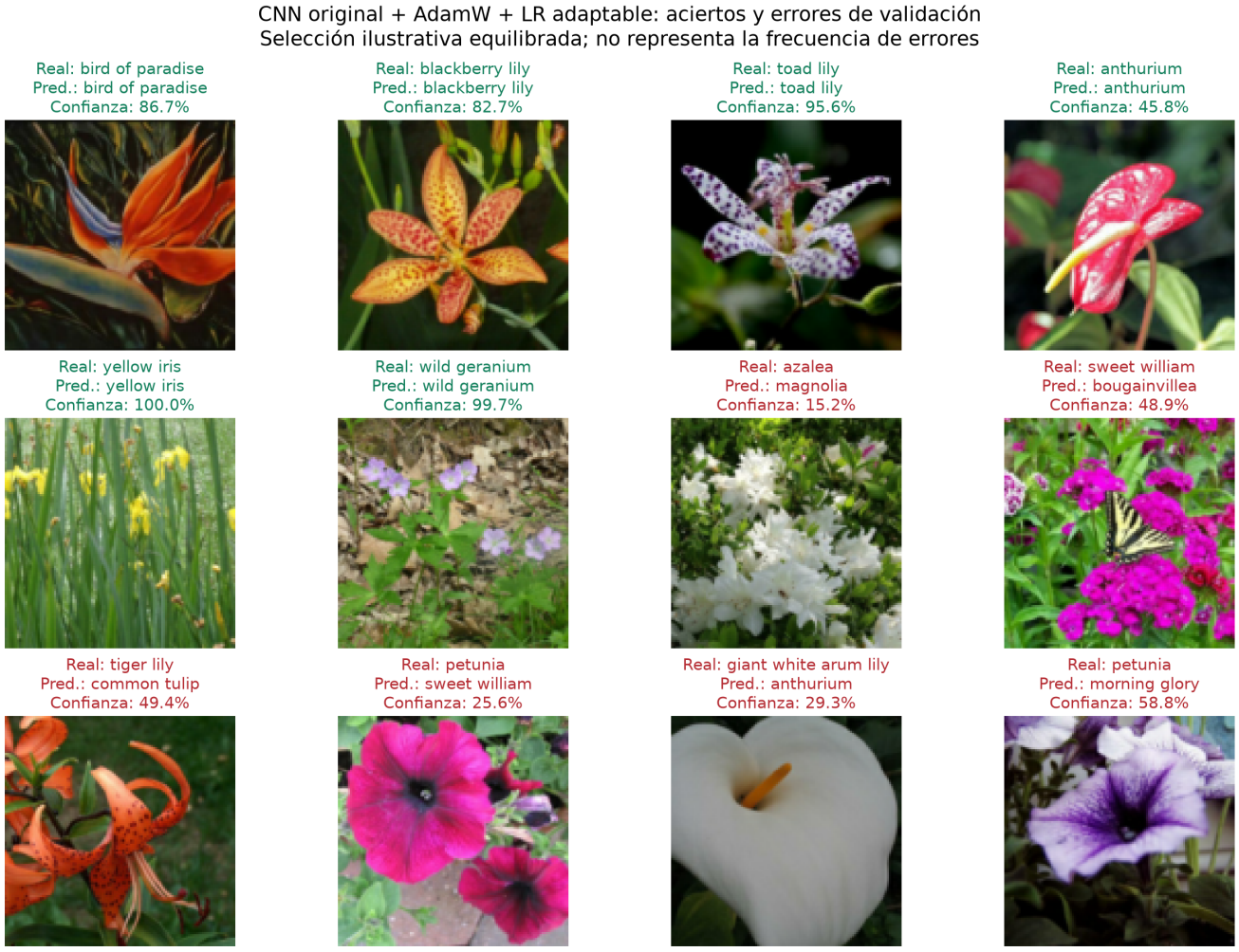


Figura 10: Ejemplos reales de validación con etiqueta verdadera, predicción y confianza softmax. El conjunto mostrado equilibra aciertos y errores solo para facilitar la inspección visual.

Referencias

Bryan Clark. What Is Image Classification? | IBM — ibm.com. https://www.ibm.com/think/topics/image-classification, 2025. [Accessed 28-09-2026].

iNaturalist. iNaturalist: A community for naturalists. https://www.inaturalist.org, 2026. [Accessed 28-09-2026].

Kaggle. Petals to the Metal - Flower Classification on TPU — kaggle.com. https://www.kaggle.com/competitions/tpu-getting-started/overview, s.f. [Accessed 28-09-2026].

Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. In *2008 Sixth Indian Conference on Computer Vision, Graphics & Image Processing*, pages 722–729. IEEE, 2008. doi: 10.1109/ICVGIP.2008.47.